

Building an AI Agent from Scratch

No Magic, Just a Deterministic Loop

A practical, implementation-first guide to understanding what an agent runtime actually does: **reason** → **choose a tool** → **execute** → **observe** → **continue or stop**.

Author: Chandan Kumar

Focus: Python · LLMs · Tool Calling · Agent Runtime Design · Reliability

Core idea

An AI agent does not need a mysterious framework to become an agent. At its simplest, an agent is a controlled loop around a language model. The model proposes the next action; deterministic application code validates and executes that action; the result becomes the next observation.

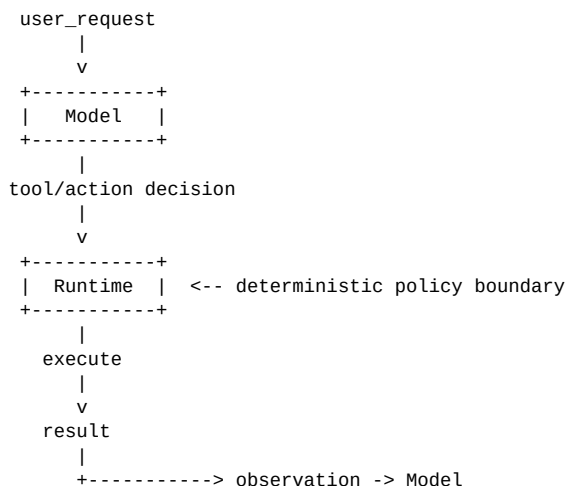
The model can decide, but your runtime decides what is allowed to happen.

What this document covers

1. Agent architecture and state
2. Tool contracts and deterministic execution
3. A complete Python agent loop
4. Working examples with calculator and weather-style tools
5. JSON tool calling without a framework
6. Validation, retries, idempotency, and stop predicates
7. Observability and production hardening
8. A practical path from prototype to production

1. What Is an AI Agent?

A normal LLM call looks like **input** → **model** → **output**. An agent adds an execution environment around that model.



A useful abstraction is:

```
state = {
  "messages": [...],
  "goal": "...",
  "steps": 0
}

while not stop(state):
  decision = model(state)
  action = validate(decision)
  observation = execute(action)
  state = update(state, action, observation)
```

The model is probabilistic. The loop does not have to be. Your code can enforce allowed tools, schemas, maximum steps, timeouts, authorization, idempotency, and termination.

Agent vs. chatbot

Capability	Chatbot	Agent
Produces text	Yes	Yes
Chooses actions	Limited	Yes
Calls tools	Optional	Core capability
Consumes tool results	Usually no	Yes
Repeats until goal/stop	No	Yes
Runtime policy boundary	Minimal	Essential

2. The Deterministic Agent Contract

Before writing code, define the boundary between model output and real-world execution.

A robust action should have a small, explicit shape:

```
{
  "type": "tool_call",
  "tool": "calculator",
  "arguments": {
    "expression": "125 * 1.18"
  }
}
```

The runtime should never blindly execute arbitrary model-generated Python, shell commands, SQL, HTTP requests, or filesystem operations. Instead, map a small set of tool names to trusted functions.

```
TOOL_REGISTRY = {
    "calculator": calculator,
    "get_weather": get_weather,
}

ALLOWED_TOOLS = set(TOOL_REGISTRY)

def validate_action(action):
    if action["type"] != "tool_call":
        raise ValueError("Unsupported action type")

    tool = action["tool"]
    if tool not in ALLOWED_TOOLS:
        raise ValueError(f"Tool not allowed: {tool}")

    if not isinstance(action.get("arguments"), dict):
        raise ValueError("arguments must be an object")

    return action
```

This registry becomes a security and reliability boundary. The LLM may request *calculator*; it does not get to invent a new executable function.

State you should keep

```
state = {
    "goal": user_request,
    "messages": [],
    "step": 0,
    "max_steps": 8,
    "last_action": None,
    "last_observation": None,
    "status": "running"
}
```

Keep the state explicit. Hidden mutable state makes agent failures much harder to reproduce.

3. Working Example: Tools Without a Framework

Start with ordinary Python functions. The agent runtime can expose them through a registry.

Safe calculator + demo weather tool

```
import ast
import operator as op

OPS = {
    ast.Add: op.add,
    ast.Sub: op.sub,
    ast.Mult: op.mul,
    ast.Div: op.truediv,
}

def calculator(expression: str) -> float:
    tree = ast.parse(expression, mode="eval")

    def visit(node):
        if isinstance(node, ast.Expression):
            return visit(node.body)
        if isinstance(node, ast.Constant) and isinstance(
            node.value, (int, float)
        ):
            return node.value
        if isinstance(node, ast.BinOp) and type(node.op) in OPS:
            return OPS[type(node.op)](
                visit(node.left), visit(node.right)
            )
        raise ValueError("Unsupported expression")

    return visit(tree)

def get_weather(city: str) -> dict:
    # Demo tool. Replace with a real provider in production.
    fake = {
        "Pune": {"temperature_c": 27, "condition": "Cloudy"},
        "Delhi": {"temperature_c": 31, "condition": "Sunny"},
    }
    return fake.get(city, {
        "temperature_c": None,
        "condition": "Unknown"
    })

TOOLS = {
    "calculator": calculator,
    "get_weather": get_weather,
}
```

The weather function is deterministic in this demo. Real APIs introduce latency, authentication, rate limits, transient errors, and changing data. The runtime must handle those separately from model reasoning.

Tool schemas

```
TOOL_SCHEMAS = {
    "calculator": {
        "description": "Evaluate simple arithmetic.",
        "parameters": {
            "type": "object",
            "properties": {
                "expression": {"type": "string"}
            },
        },
        "required": ["expression"]
    },
    "get_weather": {
        "description": "Get current demo weather for a city.",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string"}
            },
        },
        "required": ["city"]
    }
}
```

4. The Agent Loop

The loop is the heart of the system. A model response is treated as data. Your runtime parses, validates, executes, and records it.

```
def run_agent(goal, model, max_steps=8):
    state = {
        "goal": goal,
        "messages": [{"role": "user", "content": goal}],
        "step": 0,
        "status": "running",
    }

    while state["step"] < max_steps:
        state["step"] += 1

        decision = model(
            messages=state["messages"],
            tools=TOOL_SCHEMAS,
        )

        if decision["type"] == "final":
            state["status"] = "completed"
            return decision["answer"]

        action = validate_action(decision)

        try:
            result = execute_tool(action)
        except Exception as exc:
            result = {
                "ok": False,
                "error": type(exc).__name__,
                "message": str(exc),
            }

        state["messages"].append({
            "role": "assistant",
            "content": decision
        })
        state["messages"].append({
            "role": "tool",
            "name": action["tool"],
            "content": result
        })

    state["status"] = "max_steps"
    raise RuntimeError("Agent stopped: maximum steps reached")
```

Tool execution

```
def execute_tool(action):
    tool = action["tool"]
    args = action["arguments"]

    fn = TOOLS[tool]
    result = fn(**args)

    return {"ok": True, "result": result}
```

The critical property is that **execute_tool()** is ordinary deterministic application code. The model cannot bypass it.

5. Simulating the Model: A Fully Runnable Demo

You can understand the complete workflow without an API key by replacing the model with a deterministic policy. This makes the runtime testable.

```
def demo_model(messages, tools):
    user_text = messages[0]["content"]

    if len(messages) == 1:
        if "weather" in user_text.lower():
            return {
                "type": "tool_call",
                "tool": "get_weather",
                "arguments": {"city": "Pune"}
            }

        return {
            "type": "tool_call",
            "tool": "calculator",
            "arguments": {"expression": "125 * 1.18"}
        }

    last_tool = messages[-1]

    return {
        "type": "final",
        "answer": f"Tool result: {last_tool['content']}"
    }

print(run_agent("Calculate 125 * 1.18", demo_model))
print(run_agent("What is the weather in Pune?", demo_model))
```

Expected behavior

```
Tool result: {'ok': True, 'result': 147.5}
```

```
Tool result: {
  'ok': True,
  'result': {
    'temperature_c': 27,
    'condition': 'Cloudy'
  }
}
```

This is the same control flow used with a real LLM. Only the **decision function** changes.

6. Connecting a Real LLM

With a real model, the runtime asks the model for either a final answer or a structured tool call. The exact SDK varies by provider, but the architectural boundary stays the same.

```
def model_adapter(messages, tools):
    response = llm.chat(
        messages=messages,
        tools=tools,
        response_format="json"
    )

    return normalize_model_response(response)
```

A normalized response might be:

```
{
  "type": "tool_call",
  "tool": "calculator",
  "arguments": {
    "expression": "20 * 1.18"
  }
}
```

Then the loop remains unchanged:

```
answer = run_agent(
    goal="Calculate tax on 20 at 18%",
    model=model_adapter,
    max_steps=6
)
```

Provider-specific code lives in the adapter. Tool execution, authorization, logging, retry policy, and termination stay under your control.

Structured output is preferable to asking the model to emit free-form instructions. JSON is not automatically safe, but it gives the runtime something concrete to validate.

7. Multi-Step Reasoning: Tool → Observation → Tool

An agent becomes more interesting when one tool result determines the next action.

Example goal: calculate a final price after applying 18% tax, then convert it using an exchange-rate tool.

```
User
|
v
Model
|
| calculator("100 * 1.18")
v
Runtime
|
v
118
|
v
Model
|
| currency_convert(118, "USD", "INR")
v
Runtime
|
v
INR result
|
v
Model
|
v
Final answer
```

Each tool result becomes an observation in state. The next model decision sees that observation and can choose the next action.

```
while True:
    decision = model_adapter(messages, TOOL_SCHEMAS)

    if decision["type"] == "final":
        return decision["answer"]

    action = validate_action(decision)

    if not policy.allows(action, state):
        return "I cannot perform that action."

    observation = execute_tool(action)

    messages.extend([
        {"role": "assistant", "content": decision},
        {"role": "tool", "content": observation},
    ])
```

The model proposes; the runtime enforces. That distinction scales from toy examples to production agents.

8. Reliability: Retries and Idempotency

A dropped network response creates a dangerous ambiguity: did the tool fail, or did it succeed but the response disappear?

For reads, retrying is often straightforward. For writes, blindly retrying can duplicate side effects.

```
def execute_with_idempotency(action, request_id):
    key = f"{request_id}:{action['tool']}"

    if store.has_result(key):
        return store.get_result(key)

    result = execute_tool(action)

    # Persist before returning to the caller.
    store.save_result(key, result)

    return result
```

A production write tool should ideally accept an idempotency key. The runtime can generate one per logical action and reuse it during safe retries.

Retry policy

```
MAX_RETRIES = 3

for attempt in range(MAX_RETRIES):
    try:
        return execute_tool(action)
    except TransientError:
        if attempt == MAX_RETRIES - 1:
            raise
        sleep(backoff(attempt))
```

Do not classify every exception as transient. Authentication failures, invalid arguments, authorization failures, and business-rule violations generally need different handling.

9. Guardrails and Stop Predicates

An agent needs explicit stopping rules. Do not rely on the model to decide when it has spent enough time.

```
def should_stop(state):
    if state["step"] >= state["max_steps"]:
        return True

    if state.get("status") in {"completed", "failed"}:
        return True

    if state.get("budget_usd", 0) >= state["max_budget_usd"]:
        return True

    return False
```

Useful runtime limits

```
MAX_STEPS = 8
MAX_TOOL_CALLS = 12
MAX_RUNTIME_SECONDS = 45
MAX_MODEL_TOKENS = 12000
MAX_COST_USD = 0.25
MAX_TOOL_PAYLOAD_BYTES = 64_000
```

Authorization should also be evaluated outside the model:

```
def authorize(user, action):
    permissions = permission_service.for_user(user.id)

    return (
        action["tool"] in permissions.allowed_tools
        and action["tool"] not in permissions.denied_tools
    )
```

Prompt instructions are guidance; runtime policies are enforcement.

For high-impact tools such as payments, deletion, production deployment, or external communication, consider explicit human approval and stronger policy checks.

10. Observability: Make Every Step Explainable

When an agent fails, you should be able to reconstruct the run without guessing.

```
event = {
    "run_id": run_id,
    "step": state["step"],
    "tool": action["tool"],
    "arguments_hash": hash_args(action["arguments"]),
    "started_at": now(),
    "duration_ms": duration,
    "success": result["ok"],
    "error_type": result.get("error"),
}

logger.info("agent.tool_call", extra=event)
```

Capture at least:

- run ID and parent workflow ID
- model and model version
- step number
- selected tool
- validated arguments or a redacted representation
- latency
- retry count
- success/failure
- token usage and cost
- final stop reason

Avoid logging secrets, access tokens, passwords, full payment data, or unnecessary personal information. Observability should improve reliability without becoming a data-leak mechanism.

A useful mental model is a distributed trace: **model decision** → **policy check** → **tool execution** → **observation** → **next model decision**.

11. Testing the Agent Like Software

Agent behavior is probabilistic, but many important parts of the system should be deterministic and unit-testable.

```
def test_calculator():
    assert calculator("10 + 5") == 15
    assert calculator("4 * 2") == 8

def test_unknown_tool_is_rejected():
    action = {
        "type": "tool_call",
        "tool": "delete_database",
        "arguments": {}
    }

    try:
        validate_action(action)
        assert False
    except ValueError:
        assert True

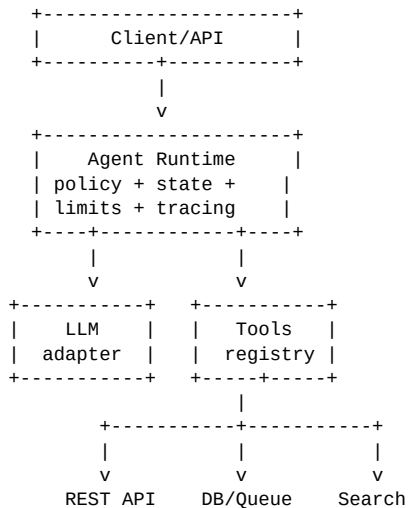
def test_max_steps():
    def looping_model(messages, tools):
        return {
            "type": "tool_call",
            "tool": "calculator",
            "arguments": {"expression": "1 + 1"}
        }

    try:
        run_agent("loop", looping_model, max_steps=2)
        assert False
    except RuntimeError:
        assert True
```

For the model itself, use evaluation datasets rather than relying only on unit tests. Measure tool selection accuracy, argument correctness, refusal behavior, latency, cost, and end-to-end task success.

Fault injection is especially valuable: simulate timeouts, malformed tool output, duplicate requests, rate limits, unauthorized actions, and model responses that reference nonexistent tools.

12. Production Architecture



Recommended boundaries

1. **Model adapter** — provider-specific API and response normalization.
2. **Runtime** — deterministic orchestration loop and state machine.
3. **Policy engine** — identity, permissions, budgets, allowed tools.
4. **Tool layer** — typed functions with explicit contracts.
5. **State store** — durable run state and idempotency records.
6. **Observability** — logs, metrics, traces, evaluations.

This architecture lets you replace the model without rewriting the execution layer, and replace tools without changing the fundamental agent loop.

13. Complete Minimal Reference Implementation

This condensed implementation captures the core pattern in one place.

```
import ast, operator as op

OPS = {
    ast.Add: op.add,
    ast.Sub: op.sub,
    ast.Mult: op.mul,
    ast.Div: op.truediv,
}

def calculator(expression):
    tree = ast.parse(expression, mode="eval")

    def visit(n):
        if isinstance(n, ast.Expression):
            return visit(n.body)
        if isinstance(n, ast.Constant) and isinstance(
            n.value, (int, float)
        ):
            return n.value
        if isinstance(n, ast.BinOp) and type(n.op) in OPS:
            return OPS[type(n.op)](
                visit(n.left), visit(n.right)
            )
        raise ValueError("unsupported expression")

    return visit(tree)

TOOLS = {"calculator": calculator}

def validate(action):
    if action.get("type") != "tool_call":
        raise ValueError("invalid action")
    if action.get("tool") not in TOOLS:
        raise ValueError("tool not allowed")
    if not isinstance(action.get("arguments"), dict):
        raise ValueError("invalid arguments")
    return action

def run(goal, model, max_steps=5):
    messages = [{"role": "user", "content": goal}]

    for step in range(max_steps):
        decision = model(messages, list(TOOLS))

        if decision["type"] == "final":
            return decision["answer"]

        action = validate(decision)
        result = TOOLS[action["tool"]](**action["arguments"])

        messages.append(
            {"role": "assistant", "content": decision}
        )
        messages.append(
            {"role": "tool", "content": result}
        )

    raise RuntimeError("max steps exceeded")
```

Even this tiny program contains the essential agent mechanics: a model decision, validation, tool execution, observation, state update, and a hard stop.

14. Practical Workflow: From 0 → Production

Stage	Build	Exit criteria
1	Single tool + deterministic model	Loop works end-to-end
2	Real LLM adapter	Structured decisions parse reliably
3	Schemas + validation	Bad actions are rejected
4	Policy layer	Unauthorized tools cannot execute
5	Retries + idempotency	Transient failures are safe
6	Tracing + metrics	Runs are diagnosable
7	Evaluation suite	Quality is measurable
8	Durable state	Runs can resume safely
9	Human approval	High-impact actions are controlled

Do not start with multi-agent orchestration, long-term memory, dozens of tools, or complex graph frameworks. First make one deterministic loop reliable.

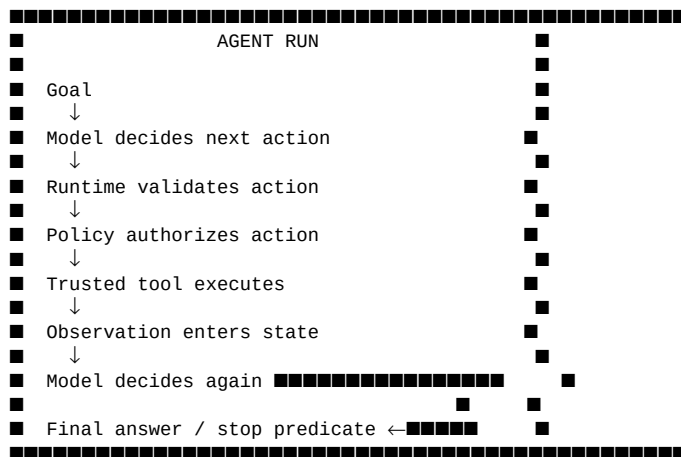
Once the core loop is solid, frameworks become productivity layers rather than abstractions you depend on to understand what your system is doing.

Key design checklist

- Explicit state
- Typed tool schemas
- Tool allowlist
- Authorization outside the model
- Maximum steps and runtime budget
- Timeouts and bounded retries
- Idempotency for writes
- Structured observations
- Traceable run IDs
- Evaluation dataset
- Human approval for high-impact actions

15. Final Mental Model

Strip away the framework names and the architecture becomes simple:



The model provides flexible reasoning. The runtime provides deterministic control.

If a tool can create side effects, the runtime should own the guardrails around that side effect. If a task can loop forever, the runtime should own the stop condition. If a request can be retried, the runtime should own idempotency. If an action requires permission, the runtime should enforce it.

One sentence to remember: An AI agent is not magic. It is a model inside a controlled execution loop.

Chandan Kumar

Full-Stack Principal Engineer | AI · LLM · RAG Pipelines · AWS · Java · Node.js · LangGraph

LinkedIn: [linkedin.com/in/chandantechie](https://www.linkedin.com/in/chandantechie)